Microservices, containers, and orchestration

When you break up a monolith into microservices, or start building a microservice-based system from scratch, you end up with a lot of services. You need to package, deploy, upgrade, and configure all those microservices. Containers address the packaging problem. Without containers, it is very difficult to scale microservice-based systems. As the number of microservices in the system grows orchestrating, the various containers and optimal scheduling requires a dedicated solution. This is where Kubernetes excels. The future of distributed systems is more microservices, packaged into more containers, which require Kubernetes to manage them. I say Kubernetes here, because in 2019, Kubernetes won the container orchestration wars.

Another aspect of many microservices is that they need to communicate with each other over a network. Where as in a monolith, most interactions are just function calls, in a microservice environment, a lot of interactions require hitting an endpoint or making a remote procedure call. Enter gRPC.